

Exercise Sheet 2, 2026

6.0 VU Advanced Database Systems

SERHII SLUTU (12537831)

May 12, 2026

Exercise 1 (Distributed Joins) [3 points]

Setup

We have a distributed database with 4 sites and three relations:

Relation	Site	# Records	Record Size (bytes)
Authors	1	4 000	450
Books	2	50 000	400
Libraries	3	10 000	750

Relevant attribute sizes (bytes):

Relation	Attribute	Size
Authors	authid	12
Authors	name	58
Books	author	10
Books	title	30
Books	library	10
Libraries	libid	10

Query 1:

$$\pi_{\text{name, title}}(Authors \bowtie_{\text{authid=author}} Books)$$

Result desired at **Site 4**. Join selectivity: $\frac{1}{9500}$.

Part (a) — Communication costs without early projection

Assumptions and cardinality estimates:

- All records and attributes are fixed size.
- Join result size: $|Authors \bowtie Books| = |Authors| \times |Books| \times \frac{1}{9500} = 4000 \times 50000 / 9500 \approx 21,053$ rows.
- Result record size (name + title): $58 + 30 = 88$ bytes. Total result size: $21,053 \times 88 = 1,852,664$ bytes.
- Semi-join estimate: $|A \ltimes B| = \min\{|A \bowtie B|, |A|\}$.
- Communication cost = bytes transferred over the network (local computation is free).

Strategy 1 — Send both tables to Site 4, join there:

$$\underbrace{4000 \times 450}_{Authors \rightarrow S4} + \underbrace{50000 \times 400}_{Books \rightarrow S4} = 1,800,000 + 20,000,000 = \mathbf{21,800,000} \text{ bytes}$$

Strategy 2 — Send *Authors* to Site 2, join there, send result to Site 4:

$$\underbrace{4000 \times 450}_{Authors \rightarrow S2} + \underbrace{21,053 \times 88}_{\text{result} \rightarrow S4} = 1,800,000 + 1,852,664 = \mathbf{3,652,664} \text{ bytes}$$

Strategy 3 — Send *Books* to Site 1, join there, send result to Site 4:

$$\underbrace{50000 \times 400}_{Books \rightarrow S1} + \underbrace{21,053 \times 88}_{result \rightarrow S4} = 20,000,000 + 1,852,664 = \mathbf{21,852,664} \text{ bytes}$$

Strategy 4 — Semi-join: send *Authors*.authid to Site 2, filter *Books*, ship back to Site 1, join, result to Site 4:

1. Send $\pi_{\text{authid}}(\textit{Authors})$ from S1 to S2: $4000 \times 12 = 48,000$ bytes.
2. $\textit{Books} \bowtie \textit{Authors}$: $\min(21,053, 50,000) = 21,053$ Books rows survive. Send back to S1: $21,053 \times 400 = 8,421,200$ bytes.
3. Full join on S1; send result to S4: $21,053 \times 88 = 1,852,664$ bytes.

$$48,000 + 8,421,200 + 1,852,664 = \mathbf{10,321,864} \text{ bytes}$$

Strategy 5 — Semi-join in opposite direction: send *Books*.author to Site 1, filter *Authors*, ship back to Site 2, join, result to Site 4:

1. Send $\pi_{\text{author}}(\textit{Books})$ from S2 to S1: $50,000 \times 10 = 500,000$ bytes.
2. $\textit{Authors} \bowtie \textit{Books}$: $\min(21,053, 4,000) = 4,000$ Authors rows survive (the join result exceeds $|\textit{Authors}|$, so all Authors are retained). Send back to S2: $4,000 \times 450 = 1,800,000$ bytes.
3. Full join on S2; send result to S4: $21,053 \times 88 = 1,852,664$ bytes.

$$500,000 + 1,800,000 + 1,852,664 = \mathbf{4,152,664} \text{ bytes}$$

Summary for part (a):

Strategy	Cost (bytes)	Cost (MB)
1 — Both to S4	21,800,000	≈ 20.79
2 — Authors to S2 ✓	3,652,664	≈ 3.48
3 — Books to S1	21,852,664	≈ 20.84
4 — Semi-join A→B	10,321,864	≈ 9.84
5 — Semi-join B→A	4,152,664	≈ 3.96

Minimum: Strategy 2 at $\approx 3,652,664$ bytes.

Note: the semi-join strategies do not help here because the join selectivity is not low enough. The join result ($\approx 21,053$ rows) is larger than $|\textit{Authors}|$ (4,000), so Strategy 5 retains all Authors anyway. Strategy 4 ships 42% of *Books* back, which is far more expensive than simply sending the small *Authors* table to *Books* (Strategy 2).

Part (a*) — Communication costs with early projection

Since Query 1 only needs **name** and **title** in the result, we project each relation to the minimum columns needed before any transfer:

- $\pi_{\text{authid}, \text{name}}(\textit{Authors})$: $12 + 58 = 70$ bytes/row, total $4000 \times 70 = 280,000$ bytes.
- $\pi_{\text{author}, \text{title}}(\textit{Books})$: $10 + 30 = 40$ bytes/row, total $50,000 \times 40 = 2,000,000$ bytes.
- Result record: $58 + 30 = 88$ bytes (unchanged). $21,053 \times 88 = 1,852,664$ bytes.

Strategy 1*:

$$280,000 + 2,000,000 = \mathbf{2,280,000} \text{ bytes}$$

Strategy 2*:

$$280,000 + 1,852,664 = \mathbf{2,132,664} \text{ bytes}$$

Strategy 3*:

$$2,000,000 + 1,852,664 = \mathbf{3,852,664} \text{ bytes}$$

Strategy 4* (semi-join, project Books before sending back):

1. $\pi_{\text{authorid}}(\text{Authors}) \rightarrow \text{S2}$: 48,000 bytes.
2. $\pi_{\text{author,title}}(\text{Books} \bowtie \text{Authors}) \rightarrow \text{S1}$: $21,053 \times 40 = 842,120$ bytes.
3. Result $\rightarrow \text{S4}$: 1,852,664 bytes.

$$48,000 + 842,120 + 1,852,664 = \mathbf{2,742,784} \text{ bytes}$$

Strategy 5* (semi-join, project Authors before sending back):

1. $\pi_{\text{author}}(\text{Books}) \rightarrow \text{S1}$: 500,000 bytes.
2. $\pi_{\text{authorid,name}}(\text{Authors} \bowtie \text{Books}) \rightarrow \text{S2}$: $4,000 \times 70 = 280,000$ bytes.
3. Result $\rightarrow \text{S4}$: 1,852,664 bytes.

$$500,000 + 280,000 + 1,852,664 = \mathbf{2,632,664} \text{ bytes}$$

Summary for part (a*):

Strategy	Cost (bytes)	Cost (MB)
1* — Both to S4	2,280,000	≈ 2.17
2* — Authors to S2 ✓	2,132,664	≈ 2.03
3* — Books to S1	3,852,664	≈ 3.67
4* — Semi-join A $\rightarrow$ B	2,742,784	≈ 2.61
5* — Semi-join B $\rightarrow$ A	2,632,664	≈ 2.51

Minimum: Strategy 2* at $\approx 2,132,664$ bytes.

Early projection significantly reduces all costs. The most dramatic improvement is in strategies that transfer *Books* (from 20 MB down to 2 MB for the full table, and from 8.4 MB down to 842 KB for the semi-join result in Strategy 4). Strategy 2* still wins: sending the small projected *Authors* to Site 2 (280 KB) is cheaper than any alternative.

Part (b) — Three-way join with Libraries**Query 2:**

$$\text{Libraries} \bowtie_{\text{libid=library}} \pi_{\text{name, library}}(\text{Authors} \bowtie_{\text{authorid=author}} \text{Books})$$

Join selectivity for the outer join (*Libraries* with inner result): $\frac{1}{4000}$.

Inner join projection for Query 2:

The inner join now needs `library` from *Books* (to join with *Libraries*) and `name` from *Authors*. So:

- $\pi_{\text{authorid,name}}(\text{Authors})$: 70 bytes/row (unchanged).
- $\pi_{\text{author, library}}(\text{Books})$: $10 + 10 = 20$ bytes/row, total $50,000 \times 20 = 1,000,000$ bytes.
- Inner result: $\pi_{\text{name, library}}(\text{Authors} \bowtie \text{Books}) = 21,053 \text{ rows} \times (58 + 10) = 68 \text{ bytes} = 1,431,604 \text{ bytes}$.
- Outer result: $21,053 \times 10,000 \times \frac{1}{4000} \approx 52,633 \text{ rows (small)}$.

Key observation for skipping strategies:

The full *Libraries* table costs $10,000 \times 750 = 7,500,000$ bytes to transfer. Any strategy that ships *Libraries* to another site will incur at least 7.5 MB just for that step. Since Strategy 1' below

costs only $\approx 1,280,000$ bytes total, all strategies involving moving *Libraries* can be immediately eliminated.

Since *Libraries* is at Site 3, the optimal approach is to bring the inner join result to Site 3. The total cost for any strategy is:

$$\text{inner join cost} + \text{cost to ship inner result to S3} + \text{small (result to S4)}$$

The inner result size is fixed at 1,431,604 bytes regardless of where it is computed. The exception is if we ship both *Authors* and *Books* directly to Site 3, avoiding a separate "inner result transfer" step entirely.

Strategy 1' — Send *Authors* and *Books* directly to Site 3, join everything there:

$$\underbrace{4000 \times 70}_{\text{Authors} \rightarrow \text{S3}} + \underbrace{50,000 \times 20}_{\text{Books} \rightarrow \text{S3}} = 280,000 + 1,000,000 = \mathbf{1,280,000} \text{ bytes}$$

(Both inner join and outer join are computed locally on Site 3. Result goes to Site 4: negligible.)

Strategy 2' — Send *Authors* to Site 2, compute inner join there, send result to Site 3:

$$\underbrace{280,000}_{\text{Authors} \rightarrow \text{S2}} + \underbrace{1,431,604}_{\text{inner result} \rightarrow \text{S3}} = \mathbf{1,711,604} \text{ bytes}$$

Strategy 4' — Semi-join (as in 4*), compute inner at S1, send result to Site 3:

$$48,000 + 21,053 \times 20 + 1,431,604 = 48,000 + 421,060 + 1,431,604 = \mathbf{1,900,664} \text{ bytes}$$

Strategies 3', 5' are skipped: Strategy 3' requires sending *Books* (1 MB) to S1 plus the inner result (1.43 MB) to S3, totalling ≈ 2.43 MB — worse than Strategy 1'. Strategy 5' requires the *Books* key transfer (500 KB) which alone is larger than the advantage over Strategy 1'.

Summary for part (b):

Strategy	Cost (bytes)	Cost (MB)
1' — Both to S3 ✓	1,280,000	≈ 1.22
2' — Authors to S2, result to S3	1,711,604	≈ 1.63
4' — Semi-join, result to S3	1,900,664	≈ 1.81
3', 5' (skipped)	$\geq 2,430,664$	≥ 2.32
Any strategy moving Libraries	$\geq 7,500,000$	≥ 7.15

Minimum: Strategy 1' — send both projected *Authors* and *Books* directly to Site 3, perform all joins locally there, then ship the small final result to Site 4. Total cost: $\approx 1,280,000$ bytes.

The key insight: for three-way joins it can be cheaper to ship all smaller tables to the site of the largest table (Site 3 for *Libraries*) rather than first computing an intermediate result and then transferring it.

Exercise 2 (Cost of MapReduce) [2 points]**Part (a) — Trace through the algorithm with $k = 2$**

The user-item interaction graph has the following edges (from Figure in the exercise sheet):

Edges:

$$u_1 \rightarrow \{i_1, i_2, i_3\}, \quad u_2 \rightarrow \{i_2, i_3, i_4\}, \quad u_3 \rightarrow \{i_3, i_4, i_5\}, \quad u_4 \rightarrow \{i_4, i_5\}$$

Mapper 1 output (emit (v, u) for each edge (u, v)):

Key	Value
i_1	u_1
i_2	u_1
i_2	u_2
i_3	u_1
i_3	u_2
i_3	u_3
i_4	u_2
i_4	u_3
i_4	u_4
i_5	u_3
i_5	u_4

Reducer 1 output (identity reducer — grouped by key):

Key	Values
i_1	$[u_1]$
i_2	$[u_1, u_2]$
i_3	$[u_1, u_2, u_3]$
i_4	$[u_2, u_3, u_4]$
i_5	$[u_3, u_4]$

Mapper 2 output (emit $((u_a, u_b), 1)$ for every pair $u_a < u_b$ sharing item v):

- i_1 : $[u_1]$ — no pairs.
- i_2 : $[u_1, u_2]$ — pair (u_1, u_2) .
- i_3 : $[u_1, u_2, u_3]$ — pairs (u_1, u_2) , (u_1, u_3) , (u_2, u_3) .
- i_4 : $[u_2, u_3, u_4]$ — pairs (u_2, u_3) , (u_2, u_4) , (u_3, u_4) .
- i_5 : $[u_3, u_4]$ — pair (u_3, u_4) .

Key (pair)	Value
(u_1, u_2)	1
(u_1, u_2)	1
(u_1, u_3)	1
(u_2, u_3)	1
(u_2, u_3)	1
(u_2, u_4)	1
(u_3, u_4)	1
(u_3, u_4)	1

Reducer 2 output (sum the values per pair):

Key (pair)	Score	(common items)
(u_1, u_2)	2	$(CI = \{i_2, i_3\})$
(u_1, u_3)	1	$(CI = \{i_3\})$
(u_2, u_3)	2	$(CI = \{i_3, i_4\})$
(u_2, u_4)	1	$(CI = \{i_4\})$
(u_3, u_4)	2	$(CI = \{i_4, i_5\})$

Mapper 3 output (emit $(u_a, (u_b, s))$ and $(u_b, (u_a, s))$ for each pair):

Key (user)	Value (candidate, score)
u_1	$(u_2, 2)$
u_2	$(u_1, 2)$
u_1	$(u_3, 1)$
u_3	$(u_1, 1)$
u_2	$(u_3, 2)$
u_3	$(u_2, 2)$
u_2	$(u_4, 1)$
u_4	$(u_2, 1)$
u_3	$(u_4, 2)$
u_4	$(u_3, 2)$

Reducer 3 output ($k = 2$: keep top-2 most similar users per user):

- u_1 : candidates = $\{(u_2, 2), (u_3, 1)\}$ — both kept (only 2 candidates).
Output: $((u_1, u_2), 2), ((u_1, u_3), 1)$.
- u_2 : candidates = $\{(u_1, 2), (u_3, 2), (u_4, 1)\}$ — top-2 are u_1 and u_3 (both score 2).
Output: $((u_2, u_1), 2), ((u_2, u_3), 2)$.
- u_3 : candidates = $\{(u_1, 1), (u_2, 2), (u_4, 2)\}$ — top-2 are u_2 and u_4 (both score 2).
Output: $((u_3, u_2), 2), ((u_3, u_4), 2)$.
- u_4 : candidates = $\{(u_2, 1), (u_3, 2)\}$ — both kept (only 2 candidates).
Output: $((u_4, u_3), 2), ((u_4, u_2), 1)$.

Key (user, neighbor)	Score
(u_1, u_2)	2
(u_1, u_3)	1
(u_2, u_1)	2
(u_2, u_3)	2
(u_3, u_2)	2
(u_3, u_4)	2
(u_4, u_3)	2
(u_4, u_2)	1

Part (b) — Cost analysis per stage

We express costs in terms of graph properties. Let $|U|$ = number of users, $|V|$ = number of items, $|E|$ = number of edges (user-item interactions). $\deg(v)$ = degree of item v (number of users connected to it). $CI(u_1, u_2)$ = set of common items of users u_1, u_2 . $S(u)$ = set of users sharing at least one item with u .

Stage 1 (Mapper 1 / Reducer 1):

- **Communication cost:** $|E|$ — each edge (u, v) is emitted exactly once as (v, u) . The identity reducer passes all pairs through unchanged.

- **Replication rate:** 1 — each edge produces exactly one key-value pair in the mapper output.
- **Reducer size:** $\deg(v)$ for the reducer handling item v (it receives one value per user connected to v). Maximum reducer size = $\max_{v \in V} \deg(v)$.

Stage 2 (Mapper 2 / Reducer 2):

- **Communication cost:** $\sum_{v \in V} \binom{\deg(v)}{2} = \sum_{v \in V} \frac{\deg(v)(\deg(v) - 1)}{2}$ — for each item v , all user pairs that share v are emitted.
- **Replication rate:** $\frac{\sum_{v \in V} \binom{\deg(v)}{2}}{|E|}$ — mapper input size is $|E|$ (output of Stage 1), mapper output is $\sum_v \binom{\deg(v)}{2}$ pairs.
- **Reducer size:** $|CI(u_1, u_2)|$ for the reducer handling pair (u_1, u_2) — it receives one 1 per common item of u_1 and u_2 . Maximum reducer size = $\max_{u_1, u_2} |CI(u_1, u_2)|$.

Stage 3 (Mapper 3 / Reducer 3):

- Let P = number of unique user pairs with at least one common item (number of edges in the similarity graph before top- k pruning).
- **Communication cost:** $2P$ — each pair (u_1, u_2, s) is emitted twice: once keyed by u_1 and once by u_2 . Equivalently, $P = |\{(u_1, u_2) : CI(u_1, u_2) \neq \emptyset\}|$.
- **Replication rate:** 2 — each pair from Reducer 2 generates exactly 2 mapper outputs.
- **Reducer size:** $|S(u)|$ for the reducer handling user u — it receives one candidate per user sharing at least one item with u . Maximum reducer size = $\max_{u \in U} |S(u)|$.

Exercise 3 (MapReduce Algorithms) [2 points]

Part (a) — MapReduce for $F_1(X, Y, Z)$

$$F_1(X, Y, Z) = \gamma_b^{SUM(a)}(\sigma_{a>b}(X) - \pi_{a,b}(Y \bowtie Z))$$

Relations: $X(a, b)$, $Y(a, b, c)$, $Z(b, c, d)$. Set semantics (no duplicates).

The computation requires three logical steps:

1. Compute the semi-join $Y \bowtie Z$ (retain tuples of Y that have a matching (b, c) in Z).
2. Compute the set difference $\sigma_{a>b}(X) - \pi_{a,b}(Y \bowtie Z)$.
3. Group by b and sum a .

Round 1 — Compute $\pi_{a,b}(Y \bowtie Z)$ and prepare $\sigma_{a>b}(X)$:

Mapper:

- For each tuple $(a, b, c) \in Y$: emit key (b, c) , value (Y, a, b) .
- For each tuple $(b, c, d) \in Z$: emit key (b, c) , value (Z) .
- For each tuple $(a, b) \in X$ with $a > b$: emit key (X, a, b) , value (X, a, b) .

Reducer (for keys of the form (b, c)):

- If the value list contains at least one Z entry, emit (YsZ, a, b) for each Y entry (a, b) in the list (these are tuples from $\pi_{a,b}(Y \bowtie Z)$).
- Keys of the form (X, a, b) are passed through as-is (identity for X tuples).

Round 2 — Compute set difference and group-by aggregation:

Mapper:

- For each X -tuple (a, b) from Round 1: emit key (a, b) , value (X) .
- For each YsZ -tuple (a, b) from Round 1: emit key (a, b) , value (YsZ) .

Reducer:

- For each key (a, b) : if the value list contains X but no YsZ , emit (b, a) — this tuple is in the set difference. If both tags are present, the tuple is excluded (set difference removes it).

Round 3 — Group by b and compute $SUM(a)$:

Mapper: For each surviving tuple (b, a) from Round 2: emit key b , value a .

Reducer: For each key b : emit $(b, \sum_{\text{values}} a)$.

Part (b) — MapReduce for min-max row normalization

Given matrix M with I rows and J columns:

$$M'_{ij} = \frac{M_{ij} - \min(M_i)}{\max(M_i) - \min(M_i)}, \quad \text{with } M'_{ij} = 0 \text{ if } \max(M_i) = \min(M_i).$$

Round 1 — Compute row min and max:

Mapper: For each entry M_{ij} : emit key i , value (j, M_{ij}) .

Reducer: For each row i , collect all values $\{(j, M_{ij})\}$.

- Compute $\min_i = \min_j M_{ij}$ and $\max_i = \max_j M_{ij}$.
- Emit $(i, \min_i, \max_i)$.

Round 2 — Normalize each entry:

Mapper (input: both M and the output of Round 1):

- For each entry M_{ij} : emit key i , value (**entry**, j , M_{ij}).
- For each $(i, \min_i, \max_i)$ from Round 1: emit key i , value (**stats**, $\min_i$, $\max_i$).

Reducer: For each row i , retrieve $\min_i$ and $\max_i$ from the **stats** entry, then for each **entry** (j, M_{ij}) :

- If $\max_i = \min_i$: emit $(i, j, 0)$.
- Otherwise: emit $\left(i, j, \frac{M_{ij} - \min_i}{\max_i - \min_i}\right)$.

Exercise 4 (MapReduce in Practice) [3 points]

The Python solutions for parts (a) and (b) are implemented in the JupyterLab notebook `Exercise4.MapReduceInPractice.ipynb`, submitted alongside this report. The notebook was executed on the LBD Cluster via JupyterHub.

Part (a): A two-round MapReduce job over `posts.csv` and `comments.csv` produces triples (`user_questioned`, `user_answered`, `user_commented`). Round 1 joins posts on their IDs to link questions, answers, and comments into (`question_owner`, `answer_owner`, `comment_user`) triples. Round 2 emits the final output.

Part (b): A MapReduce job over `posts.csv` and `users.csv` computes per-user post counts and outputs the top-100 users by reputation. A combiner is used to pre-aggregate question and answer counts per user within each mapper, reducing the data sent to reducers.

Exercise 5 (Spark) [5 points]

The Spark solutions are implemented in the JupyterLab notebook `Exercise5_Spark.ipynb`, submitted alongside this report. The notebook was executed on the LBD Cluster via Jupyter-Hub.

Part (a): SQL and DataFrame translations of the given queries.

Part (b): Analysis of Spark stage boundaries and the impact of optimization blocks on `user_scores`, `tag_scores`, and `user_comment_scores`.

Part (c): Translation of the Top-K user similarity graph algorithm from Exercise 2 into PySpark RDD transformations and actions.

Part (d): PySpark RDD implementation for finding the top-K most frequently co-purchased item pairs from retail transaction data.